Prepared for submission to JHEP

dSIBP: a package for generating IBP seeds for QFT in de Sitter spacetime

a

a

Abstract: `dSIBP` is a Mathematica package that generates the integration-by-parts (IBP) relations of Feynman integrals in de Sitter (dS) spacetime. The input is a Feynman graph whose internal lines are massive or massless and live on the two Schwinger–Keldysh branches. The package builds the integral family of this graph, writes its canonical IBP seeds, and assembles them into a sealed linear system for the external reducers Kira and Rational Tracer. One further command differentiates the reduced masters and returns their differential equations, together with Euler scaling identities that check the result. A parallel track treats tree-level families, which carry no loop momentum. The package presents no new derivation; all formulas come from the existing de Sitter IBP constructions. Its usage is described through worked examples, from the sunrise graph to a mixed loop-plus-tree family with eighty-one master integrals.

Contents

1 Introduction **1**

2 Notation and conventions **4**
- 2.1 Schwinger–Keldysh Feynman rules 4
- 2.2 The integral family 6
- 2.3 Kinematics 7
- 2.4 Parity structure 8
- 2.5 Dynamical rules and massive function systems 8
- 2.6 Tree-level and time-only families 9

3 Using the package **9**
- 3.1 Initialization 9
- 3.2 Seeds and the IBP envelope 9
- 3.3 Linear systems and external reduction 10
- 3.4 Differential equations and checks 11
- 3.5 Tree-level track 12

4 Examples **12**

5 Summary and outlook **14**

A Public functions **16**

1 Introduction

Correlation functions of quantum fields in de Sitter (dS) spacetime are the basic observables of inflationary perturbation theory. They are computed in the in-in formalism, whose diagrammatic form is the Schwinger–Keldysh path integral [1]. After the Feynman rules are applied, a graph becomes scalar integrals over the conformal times $\tau_v < 0$ of its vertices. The integrand is a product of vertex phases $e^{-i\sigma_v E_v \tau_v}$ and of propagator functions. Massive internal states make these integrals hard, because their propagators are written with Hankel functions, so the integrand is no longer of polynomial type. Results with massive states are still sparse. Ref. [2] states the situation directly: explicit computations of such correlators remain largely limited to tree level and to bubble topologies at one loop.

The route taken in this paper comes from flat-spacetime QFT, where the same problem has a standard solution. Integration by parts (IBP) [3] expresses every integral of a family as a linear combination of a small set of master integrals, and Laporta's algorithm made this reduction automatic [4]. Differentiating the masters and reducing again gives a closed system of first-order linear differential equations (DE) [5, 6]. That method produced the

dimensionally regulated pentagon integrals [7] and the two-loop four-point functions [8], and it becomes much simpler when the DE are written in $d\log$ form [9].

What made this route decisive is not only the algebra but also the software built around it. Seed generation and reduction are automated by FIRE [10, 11] and by Kira [12, 13], and rational-function reconstruction by Rational Tracer [14]. The DE are solved numerically by DiffExp [15] and by the auxiliary mass flow method with its implementation AMFlow [16, 17]. This last series also shows how little input the whole scheme needs. Reference [18] finds that Feynman integrals with any number of loops are completely determined once linear relations among them are given, and it verifies this on examples up to five loops. Computing an integral then becomes linear algebra plus numerical transport, with no integration input left to supply. In practice the user writes down a topology, and a program handles the rest.

The same two steps were set up for dS space in refs. [2, 19, 20]. Reference [19] extends IBP from polynomial integrands to the Hankel integrands of dS. It proves universal formulas for the iterative reduction and for the $d\log$-form DE of an arbitrary vertex integral family, and it states that these formulas dominate all tree-level dS correlators and play a kernel role at loop level. Reference [20] solves such equations by power series, gives multivariate hypergeometric expressions for the homogeneous part of an arbitrary tree, and shows how to determine the boundary data. Reference [2] treats the loop-momentum integrals as well. It identifies a structural property of the dS IBP system: an n-propagator family splits into 2^n closed subsystems, classified by the parity of the propagator indices. Its explicit construction is carried out for the one-loop bubble family.

These works also point at the same missing layer. Reference [19] observes that existing flat-space programs can be reused, with Kira solving the IBP reduction of a user-defined system, and it leaves the boundary data of the DE outside its scope. Reference [20] notes that the automatic tools widely used in flat QFT could help. None of the three releases a program that turns a general dS graph into that user-defined system. Assembling the system by hand is laborious and error-prone already at one loop. A massive line carries three integer labels: a propagator power and two Hankel powers. Massive lines obey second-order equations of motion, so only two of their vertex states are independent. Massless same-branch lines must be treated as a quotient over vertex states. The parity structure above closes the system into sectors, and only one sector is needed per reduction.

`dSIBP` is a MATHEMATICA package that automates this step. The user declares a Feynman graph by its vertices, by the four Schwinger–Keldysh types of its internal propagators, and by its external momenta. The package then performs the following tasks.

- It validates the kinematic declarations, namely the loop momenta, the no-loop external magnitudes and their dynamical rules. It reports whether the user lists are complete or over-complete.

- It constructs the integral family J with structured labels for propagator powers, vertex states and user-defined irreducible scalar products (ISPs).

- It generates canonical IBP seed equations from the time generators at all active vertices and from the $L(L+K)$ loop-momentum generators. The massive vertex states are restricted to the proven representative set $\{0, 1\}$.
- It sweeps the continuous labels over a user-specified envelope and assembles a sealed, backend-neutral linear system.
- It exports the system to Kira [12, 13] or Rational Tracer [14] for the actual reduction, and it imports the results back.
- It differentiates the reduced master integrals with respect to the independent kinematic variables. This yields differential equations, optionally in $d\log$ form. Euler scaling identities provide an independent check of the result.

Two boundaries of the package are worth stating. It generates and serializes IBP relations, but it never runs a reduction itself. The reduction is delegated to external programs, which keeps the package independent of any particular reduction engine. The package also adds no new derivation: every formula it uses is derived in refs. [2, 19, 20]. What it supplies instead is the notation those formulas need in order to be applied to an arbitrary graph, and a machine-readable form of the system they generate. Every symbol that appears in the output of the package is traced back to its definition.

A complementary tree-level track handles time-only families, which carry no loop momentum. Its seeds can be reduced internally to differential equations and to $d\log$ forms. This track interfaces with the package MadStree [21], which evaluates tree-level cosmological correlators numerically. Related differential structures for cosmological loop integrands appear in the kinematic-flow construction [22–24], which is not implemented here.

The paper is organized as follows. Section 2 fixes the notation: the Schwinger–Keldysh Feynman rules, the integral family and its labels, the kinematic variables, the parity structure, the dynamical rules and the time-only representation. Section 3 describes the public interface, from initialization to seeds, linear systems, external reduction and differential equations, and then the tree-level track. Section 4 walks through the examples shipped with the package, including a mixed loop-plus-tree family whose reduction system holds more than eight hundred thousand relations. Section 5 closes with an outlook. Appendix A lists the public functions with one-line descriptions.

Statement on AI use. The notation, the functionality and the interfaces of `dSIBP` were designed by the author. The author supervised a number of different agent programs and large language models that implemented the code and the documentation. The author also designed the validation tasks that establish the reliability of the package. All of these tasks are cross-validations of computed results against independent data. They cover the IBP relations, the differential equations and the scaling identities as follows.

- *Massive two-vertex family.* The general IBP seeds and the differential equations are checked against the analytic results of ref. [20]. The family has five normalized masters. The three 5×5 differential-equation matrices agree entry by entry, 25/25 exact

in each matrix. The vertex basis and the ordering of the masters follow ref. [19]. This task validates the IBP relations and the differential equations.

- *Massive three-vertex tree family.* The formulas were first derived independently by an agent, which generalized an earlier hand-written two-vertex implementation. They were checked against ref. [2] and only then compared with the output of the package. This task validates the tree-level IBP relations and the tree-level differential equations.

- *One-loop bubble, full pipeline.* The IBP system is reduced by an external Kira run, which closes with no unreduced target and returns nineteen masters. The two 19×19 differential-equation matrices keep both derivative variables symbolic. At the recorded exact point they reproduce the matrices of ref. [2] entry by entry, 361/361 for each of the three variable conventions. The scaling identities hold on the reduced basis: 33/33 source-level identities and 19/19 active-basis checks pass symbolically. This task validates the IBP relations, the differential equations and the scaling identities.

- *Mixed bubble-plus-tree family, full pipeline.* The family has eighty-one master integrals. The external Kira reduction closes with no unreduced target. The six 81×81 differential-equation matrices are free of residual symbols. All 81/81 source-level Euler scaling and IBP identities vanish exactly. These identities are independent of the numerical reduction. The reduction itself carries a scaling certificate at a recorded exact point. This task validates the IBP relations, the differential equations and the scaling identities.

- *General formulas against naive sampling.* The package substitutes the derived general formulas instead of solving IBP systems again. Its output was compared with reductions of naively sampled raw IBP seeds of the same families. The IBP relations and the differential equations agree. This task validates the IBP relations and the differential equations.

- *Tree-level differential equations against MadStree.* The direct $d\log$ route and the naive IBP route of the package give the same result. In addition, for a mixed three-vertex tree family with fifteen masters, the parameter derivatives of `dSIBP` were transported into MadStree [21] for the five variables $k_1, k_2, k_3, \mathtt{sE1}, \mathtt{sE2}$. All seventy-five derivatives closed, and the five 15×15 differential-equation matrices agree entry by entry with the direct $d\log$ formula route of MadStree. This task validates the tree-level differential equations.

The package is available at https://github.com/J7C/dSIBP-Packages.

2 Notation and conventions

2.1 Schwinger–Keldysh Feynman rules

A de Sitter Feynman graph in the in-in formalism consists of vertices v with conformal time $\tau_v < 0$ and Schwinger–Keldysh branch signs $\sigma_v = \pm$. Each vertex with external energy E_v

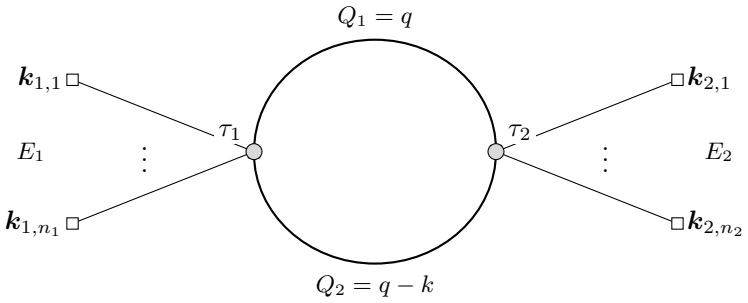


Figure 1. Feynman rules of a de Sitter bubble with general branch signs, drawn as gray vertices. The vertex energies E_1, E_2 and the conformal times τ_1, τ_2 are written at the vertices. The external legs end in open squares and carry the momenta $k_{1,1}, \ldots, k_{1,n_1}$ and $k_{2,1}, \ldots, k_{2,n_2}$. The two internal propagators carry the loop momenta q and $q - k$.

carries the phase factor

$$e^{-i\sigma_v E_v \tau_v} , \tag{2.1}$$

and the time integrations run over the full interval $\tau_v \in (-\infty, 0)$. An internal propagator e joining the vertices u and v is one of four types. It is massive or massless, and it is full (same branch, $\sigma_u = \sigma_v$) or cross (opposite branch, $\sigma_u = -\sigma_v$). The four propagator types have distinct Heaviside structures and different kernels. The massive full and cross propagators differ by a factor i, and the two massless cross propagators likewise differ by a factor i. The massless full propagator is the sum of its two ordered halves. In our figures a circle denotes a vertex, and its fill colour encodes the branch sign: black is $\sigma_v = +$, white is $\sigma_v = -$, and gray leaves the sign general. The conformal time τ_v is written below the vertex and the vertex energy beside it. External legs end in small open squares. The energy E_v of a vertex is the sum of the magnitudes of all external momenta routed into v, plus any vertex energy parameter z_v. Figure 1 shows these rules for the bubble input of the package. Its two vertices are drawn in gray because their branch signs are general. Vertex 1 carries the external momenta $k_{1,1}, \ldots, k_{1,n_1}$ and vertex 2 carries $k_{2,1}, \ldots, k_{2,n_2}$. The internal momenta are q and $q - k$.

The package distinguishes the four propagator types as $\langle$`massive, massless`$\rangle \times \langle$`full, cross`$\rangle$. Figure 2 shows the resulting graphical conventions. Every propagator that lies on a cycle becomes a denominator of the integral family. Non-cyclic propagators attached to the loop skeleton are *fixed propagators*. They carry only a magnitude power and no loop-momentum denominator. Massive propagators whose two vertices become coincident under a contact operation may be *shrunk*. Shrunk propagators survive only through a structured prefactor of the sector.

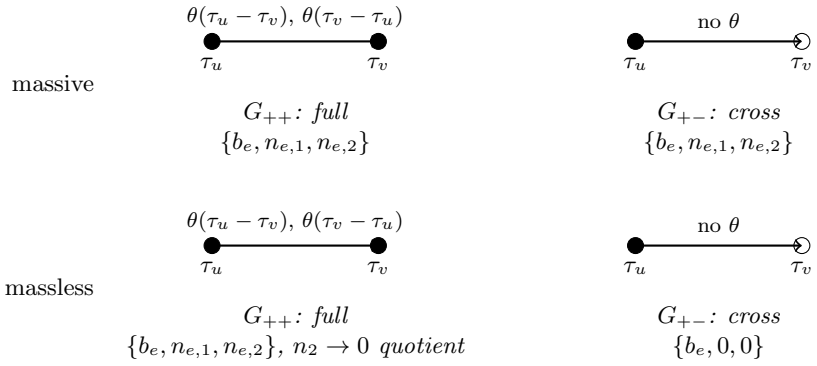


Figure 2. The four types of internal propagator and their label packs. The fill colour of a vertex encodes its branch sign: black is + and white is −. The conjugate assignments −− and −+ give the same two types. Massive and massless full propagators carry $\{b_e, n_{e,1}, n_{e,2}\}$, with the massive vertex states restricted to $\{0, 1\}$. Cross propagators carry $\{b_e, 0, 0\}$. Massless full propagators are the quotient of the two ordered vertex states, with $F_{01} = -F_{10}$ and $F_{11} = -F_{00}$.

2.2 The integral family

For a sector s, that is, a subset of propagators retained as denominators, a generic member of the integral family is a time-ordered integral over the vertex times,

$$\begin{aligned} J[s, \{a_v\}, \{B_e, n_{e,1}, n_{e,2}\}, \{\rho_r\}] = & \int_{-\infty}^{0} \prod_v d\tau_v \prod_v (-\tau_v)^{a_v + a0_v} \, e^{-i\sigma_v E_v \tau_v} \\ & \times \prod_{e \in s} \mathcal{G}_e(\tau_u, \tau_v)^{B_e} \prod_e H_e^{n_{e,1}} \, \bar{H}_e^{n_{e,2}} \prod_r \rho_r^{ir} \, . \end{aligned} \tag{2.2}$$

Here $e = (u, v)$ runs over the propagators of the sector, $\mathcal{G}_e$ is the corresponding Schwinger–Keldysh propagator function, and $H_e, \bar{H}_e$ are the Hankel-function blocks attached to the two vertices of a massive propagator. In the package the family is written in the three-slot schema

```
J[sectorKey_String, linePacks_List, ispList_List]
```

The first slot identifies the sector by the retained propagators. The second slot holds one pack per propagator, in the order in which the input key `"lines"` lists them.

- A cyclic full propagator contributes $\{b_e, n_{e,1}, n_{e,2}\}$: the propagator power b_e and the Hankel powers at the two vertices.
- A cyclic massless cross propagator contributes $\{b_e, 0, 0\}$.
- A fixed propagator contributes $\{$`"F"`$, n_{e,1}, n_{e,2}\}$: no denominator power, only the magnitude powers of the propagator.
- A shrunk propagator contributes $\{b_{S,e}\}$, the magnitude power kept by the sector prefactor.

The third slot lists the ISPs with their integer indices i_r.

The zero points of the labels are fixed by the package and cannot be changed by the user. For the h-preset dynamics the vertex time powers have $a0_v = 2\nu_{\mathrm{ref}}$, and the massive propagator powers have $b0_e = -2\nu_e$. The *actual* powers that enter the integrand are therefore $A_v = a_v + a0_v$ and $B_e = b_e + b0_e$, while the package-level labels $a_v, b_e, n_{e,i}$ run over small non-negative integers. These conventions follow refs. [2, 20]. They make the IBP shifts uniform: the generators shift the package-level labels by ± 1 and never touch the zero points.

Massive vertex states and equations of motion. A massive propagator e with mass parameter ν_e and the h-preset $P_h(x) = (2\nu_e + 1)/x$, $Q_h(x) = 1$ has Hankel blocks at each of its two vertices that span a two-dimensional space. The space is indexed by the state $n_{e,i}$. The second-order equation of motion gives, at $n_{e,i} = 2$, the exact relation

$$\begin{aligned} J\big[\cdots, n_{e,i} = 2, \cdots\big] = & -J\big[\cdots, n_{e,i} = 0, \cdots\big] \\ & -(2\nu_e + 1)\, J\big[\cdots, n_{e,i} = 1,\, a_r \to a_r - 1,\, b_e \to b_e + 1, \cdots\big], \end{aligned} \tag{2.3}$$

so every massive vertex state reduces to the representative set $\{0, 1\}$. The package applies this reduction by construction: the generated seeds contain only $n_{e,i} \in \{0, 1\}$.

Massless quotient. A massless full propagator does not distinguish its two vertices. The package fixes an ordered state $(n_{e,1}, n_{e,2})$ per propagator and works with the quotient

$$F_{01} = -F_{10}, \qquad F_{11} = -F_{00}, \tag{2.4}$$

so the public family keeps a single bit per massless full propagator, taken from the representatives F_{00} and F_{10}. This is an exact algebraic identification, not a sampling prescription.

Shrink rules. A massive propagator may be shrunk only in the `massiveFull` type and only when $n_{e,1} + n_{e,2} = 1$. The shrink multiplies the sector by the coefficient $-\chi_e(-1)^{n_{e,r}} C_e$, where $\chi_e = \pm 1$ is fixed by the two vertex types and $C_e = \frac{4i}{\pi} e^{\pi \operatorname{Im} \nu_e}$. A massless full propagator shrinks with a ∓ 2 delta term and a $+2$ term, without a Wronskian prefactor and without label shifts. If all the propagators meeting at a vertex shrink simultaneously, the odd-subset contact terms

$$\sum_{\substack{\emptyset \neq S \subseteq B \\ |S| \text{ odd}}} 2^{\,1-|S|} \prod_{e \in S} D_e \prod_{e \in S} J_e \tag{2.5}$$

are generated. Shrunk propagators never reappear as denominators. Their magnitude powers live in the structured sector prefactor.

2.3 Kinematics

The structural loop number L is the first Betti number of the internal undirected multigraph. Let E count the internal propagators, each one counted once: parallel propagators between the same pair of vertices therefore contribute separately, and a self-loop contributes

one. External legs do not contribute to E. Let V count the vertices and C the connected components. Then

$$L = E - V + C\,. \tag{2.6}$$

Together with K independent loop-related external momenta, the loop number fixes the number of scalar products,

$$N_{\mathrm{sp}} = \frac{L(L+1)}{2} + LK\,. \tag{2.7}$$

The user declares two separate lists. The first list, `loopExternalMomenta` $\{k_1, \ldots, k_K\}$, defines the loop coordinates `ss`$ij = \sqrt{\mathrm{sp}[k_i, k_j]}$ of the loop Gram matrix. The second list, `independentExternalMomenta`, defines the no-loop magnitudes $|$`kE`$i|$ that appear in vertex energies and in the masses of fixed propagators. These magnitudes are parameterized by `sE`i. Only magnitudes are coordinates. The package never outputs $\mathrm{sp}[k_{Ei}, k_{Ej}]$, and it never eliminates intermediate magnitudes by a cosine rule. Magnitudes that depend on the declared ones are stored as explicit bindings. They still contribute to vertex phases through the chain rule.

User-defined ISPs are declared as `name/expr/range` associations, for example

```
<|"name" -> rho1, "expr" -> sp[q1, k], "range" -> {0, 1}|>,
```

and they must form an invertible coordinate system on the space of scalar products together with the propagator squares. The definition zero point of an ISP is fixed at 0: positive powers are numerator powers, and a negative power means an extra denominator. Kinematic rules of the form $\mathrm{sp}[k_i, k_j] \to$ dynamical variables are validated by the two-stage interface `DSKinematics`/`DSInit`. The interface reports exact, over-complete or under-complete states. Only the exact state opens derivative generation.

2.4 Parity structure

The characteristic discrete invariant of de Sitter IBP is the parity of the sum of the labels on a propagator. For every cyclic propagator e the quantity

$$p_e \;=\; b_e + n_{e,1} + n_{e,2} \pmod 2 \tag{2.8}$$

is invariant under all loop and time generators acting on a full massive propagator. It is preserved by the h/H shrink of massive propagators. For massless full propagators it flips by construction: the contact operation maps $b_e + n_{e,1} + n_{e,2} \to b_e + 1$, so the massless parity of a shrunken sector is offset by one. Cross propagators carry no parity constraint. Every generator preserves the parity vector $(p_e)_e$. The IBP system of a family therefore splits into independent parity blocks. For families with user-provided `parityConstraints` the package restricts the seeds to the requested block, for example $b_{[1]} + n_{[1,1]} + n_{[1,2]} \equiv 0$. This reproduces the parity-split IBP system of ref. [2]. ISP indices receive no parity filter.

2.5 Dynamical rules and massive function systems

Every massive propagator is assigned a second-order function system $f_e'' + P_e(x)\, f_e' + Q_e(x)\, f_e = 0$. The default `"h"` preset uses $P_h = (2\nu_e + 1)/x$ and $Q_h = 1$. The package also accepts bare Hankel functions H and general user systems (P, Q, T, W). Let T_e be

the transport matrix of the fundamental system and W_e its Wronskian. The package tracks the combination $W_T = \det(T)\, W$ through all shrinks. The theta-coincidence and Wronskian shrink terms used by the time-IBP generators are derived from it. Ordinary derivatives of dynamical quantities never generate theta shrinks and never invoke W_T. The user chooses the convention once at initialization. The IBP generators, the derivative operators and the tree-level track all read the same compiled function system.

2.6 Tree-level and time-only families

For graphs without cycles the package exposes a `timeOnly` family with the unique public representation

```
J[sectorKey_String, timeShifts_List, stateBits_List]
```

Here `sectorKey` records, for each propagator of the root topology, whether it is full (1) or shrunk (0). The list `timeShifts` gives the time powers a_v of the active vertices. The list `stateBits` gives one bit for each massive vertex state or massless quotient state, in the frozen order of the root propagators. There are no momentum generators, no propagator powers and no ISPs. The magnitude powers of fixed propagators remain in the structured sector prefactor and still enter derivative generation. The massive-only time-only families are fully certified. A certification gate controls the remaining cases, and the public interface exposes no way around it. Families that contain massless full propagators are returned as `PendingRederivation`, and their recorded reason is `masslessQuotientFormulaNotCertified`.

3 Using the package

3.1 Initialization

A case is a named topology declaration. Initialization runs a structural audit of the loop number, the registry of vertices and propagators and the external-momentum routing. It then runs the kinematic audit of section 2.3. The result is a context that every later command consumes:

```
context = DSInit[case, KinematicRules -> custom];
```

The optional two-stage interface `DSKinematics[case]` first returns a proposal with the default rules and a selection template. The user can then inspect the kinematic coordinates `ss`ij, `sE`i, the momentum-routing audit and the capability flags before committing. Under-complete declarations abort the initialization. Over-complete ones allow symbolic seed generation but disable derivatives and the inner-form rewriting. In the over-complete case common loop shifts are quotiented out through an affine basis `effectiveLoopExternalMomenta`.

3.2 Seeds and the IBP envelope

`DSSeeds[context]` returns the canonical seed templates of the family. Every actual sector receives all active time generators and all $L(L+K)$ loop-momentum generators. The

massive vertex states are pre-reduced by the equation of motion (2.3). Massless full propagators run over two quotient representatives. The discrete n labels are eliminated, and the analytic seeds keep general continuous labels $a_v, b_e, b_{S,e}$ and general ISP indices. Each template is hash-sealed. Templates that the equation of motion and the canonicalization reduce to the identity $0 = 0$ are retained and audited rather than dropped.

The continuous labels are swept by `DSGenerateIBP`. The unified form

```
ibpData = DSGenerateIBP[allSeeds, {-1, 1}];
```

requires that every root continuous index lies in the closed integer interval $[-1, 1]$ *after* all shifts of the generated relations. The package inverts the per-index shift sets and derives the narrower seed ranges automatically. The per-index form

```
ibpData = DSGenerateIBP[allSeeds,
   {a[v1], -1, 1}, {a[v2], 0, 1},
   {b[e1], 0, 2}, {b[e2], -1, 1}];
```

must cover all root continuous indices exactly. ISP lower bounds are combined with the user target by taking the larger lower bound, so the package never extends a range on its own in order to cover numerator terms. The output is a sealed contract with status `"generated"`, a template-set hash, an integrity audit and the capability flags `sealedQ`, `completeSystemQ` and `subsetQ`. Only sealed complete systems may proceed to a formal reduction.

3.3 Linear systems and external reduction

`DSLinear[ibpData, context]` converts the sealed seeds into a backend-neutral association `linearData`. Its fields are the equation list `linearEquations`, the master-integral ordering `integralList`, the pivot rules `integralRules`, the sector metadata `sectorMetadataList` and the readiness reports. The order of `integralList` fixes the master-integral ordering. The command `DSReorderIntegrals` re-indexes it if the user prefers a different order. The two-stage Kira plan is built purely in memory:

```
prePlan   = DSKiraPlan[linearData, <|"stage" -> "preReduction", ...|>];
preExport = DSKiraExport[prePlan];
(* ... external Kira run identifies the master integrals ... *)
userMIData = DSUserMI[linearData, userExpressions, <|
   "names" -> activeNames, "activeIndices" -> activeIndices,
   "derivativeVariables" -> {ss11, P0}, "scalingDegrees" -> degrees|>];
formalPlan   = DSKiraPlan[linearWithUserMI, <|"stage" -> "formal", ...|>];
formalExport = DSKiraExport[formalPlan];
(* ... external Kira run reduces to the user basis ... *)
importedData = DSKiraImport[formalWorkspace];
```

The pre-reduction stage freezes the candidate targets and lets the external reducer discover the masters. The command `DSUserMI` then attaches the user basis. It checks the global and the active rank, and it stores reversible pivot rules. The formal stage constructs the

analytic derivative closure of the active basis *before* any export. It refuses to write if internal helper symbols survive. Export manifests carry the SHA-256 identity of every artifact, and `DSKiraImport` re-verifies the contents instead of trusting version strings. The serializer maps momenta to the imaginary axis $k \to -I\,ik$ expected by Kira. The same `linearData` feeds the Rational Tracer [14] pipeline through the equation-system interface.

3.4 Differential equations and checks

After the reduction, `DSDE` differentiates the master integrals with respect to the independent variables. Loop-Gram coordinates are handled through external-vector derivative decompositions. No-loop magnitudes are handled through propagator-local radial derivatives, including their dependent bindings. Vertex energies enter as scalar parameters. Explicit coefficients are differentiated directly, while bare integrals are shifted and canonicalized. The result is reduced back to the active basis. For the pure massive bubble the resulting equations are in $d\log$ form. At the recorded exact point they reproduce the matrices of ref. [2] entry by entry.

Every integral of the family is a homogeneous function of the dimensionful variables. Let $x_1, \ldots, x_m$ denote these variables, that is, the loop coordinates `ss`ij, the no-loop magnitudes `sE`i and the mass scales. Each master integral I_k then satisfies the Euler relation

$$\sum_{a=1}^{m} x_a \frac{\partial I_k}{\partial x_a} \;=\; d_k\, I_k\,, \tag{3.1}$$

where d_k is the constant scaling degree of I_k. In matrix form, with $\partial I/\partial x_a = A_a(x)\, I$, the identity (3.1) becomes

$$\sum_{a=1}^{m} x_a\, A_a(x) \;=\; \Delta\,, \qquad \Delta = \mathrm{diag}(d_1, \ldots, d_N)\,. \tag{3.2}$$

The command `DSScaleCheck` verifies (3.2) symbolically on the reduced basis. It also checks the source-level time and loop-momentum IBP identities before the reduction. Because Δ is constant and diagonal, (3.2) is a strong self-consistency test of the whole chain from seeds to differential equations. The tree-level track exposes `DSTreeDLogDE` for direct $d\log$ differential equations.

At the atomic level the package also exposes the generators themselves:

```
dtau[i, expr, context]     (* time generator at vertex i *)
dqq[i, j, expr, context]   (* d/dq_i . q_j *)
dqk[i, j, expr, context]   (* d/dq_i . k_j *)
ds[expr, var, context]     (* full variable derivative *)
```

together with the form rewriters `rep2innerform`, `rep2outform` and `rep2Integrand`. The last command expands a J into its explicit integrand without integrating. The short signatures rely on the single registered topology. Explicit contexts should be used when several topologies are alive.

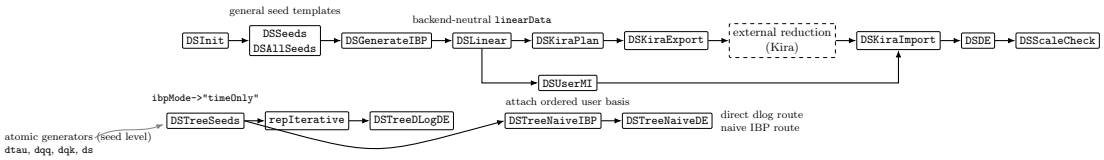


Figure 3. The public workflow of `dSIBP`. Full (loop) families run from `DSInit` through the seed, envelope and linear-system stages to the two-stage Kira export and import and to the differential-equation layer. The command `DSUserMI` attaches the user basis between the two reduction stages. Tree-level (`timeOnly`) families run through `DSTreeSeeds` and `repIterative` to `DSTreeDLogDE`, with the naive IBP route as an independent cross-check. The atomic generators `dtau`, `dqq`, `dqk` and `ds` are available at every stage.

3.5 Tree-level track

The tree-level functions mirror the loop workflow:

```
treeSeeds = DSTreeSeeds[context];
iterative = repIterative[treeSeeds, context];
naiveIBP  = DSTreeNaiveIBP[treeSeeds, context];
naiveDE   = DSTreeNaiveDE[naiveIBP, context];
dlogDE    = DSTreeDLogDE[iterative, context];
```

The iterative route substitutes the general derived formulas directly. The naive route reduces raw sampled IBP relations. The two routes must agree, and the package reports the flag `deRoutesAgree`. The tree-level differential equations have also been compared with the package MadStree [21]. MadStree evaluates tree-level cosmological correlators and builds its own $d\log$ differential equations. For a mixed three-vertex tree family with fifteen masters, the parameter derivatives of `dSIBP` were reduced by MadStree for the five variables $k_1, k_2, k_3, \mathtt{sE1}, \mathtt{sE2}$. All seventy-five derivatives closed, and the five 15×15 matrices agree entry by entry with the direct $d\log$ formula route of MadStree. Figure 3 summarizes the two tracks and their connection to the external reducers. Appendix A lists the public functions with short descriptions.

4 Examples

The package ships six worked examples. Figure 4 shows the four graphs discussed below. The quoted Kira times are wall-clock times of the external reduction on a single workstation. The Kira inputs were written by `DSKiraExport`.

Sunrise (example 03). The graph of figure 4a has two vertices joined by three parallel propagators, hence $L = 2$. Propagator 1 is a massive full propagator of mass ν_1, and propagators 2 and 3 are massless full propagators. The loop momenta are l_3 and k_{321}, and the single loop external momentum is k_L. Thus $K = 1$, and the loop Gram matrix gives the coordinate $\mathtt{ss11} = |k_L|$. No no-loop magnitude is declared: both vertices carry the same external-leg energy k_E, which is the second derivative variable. The top sector

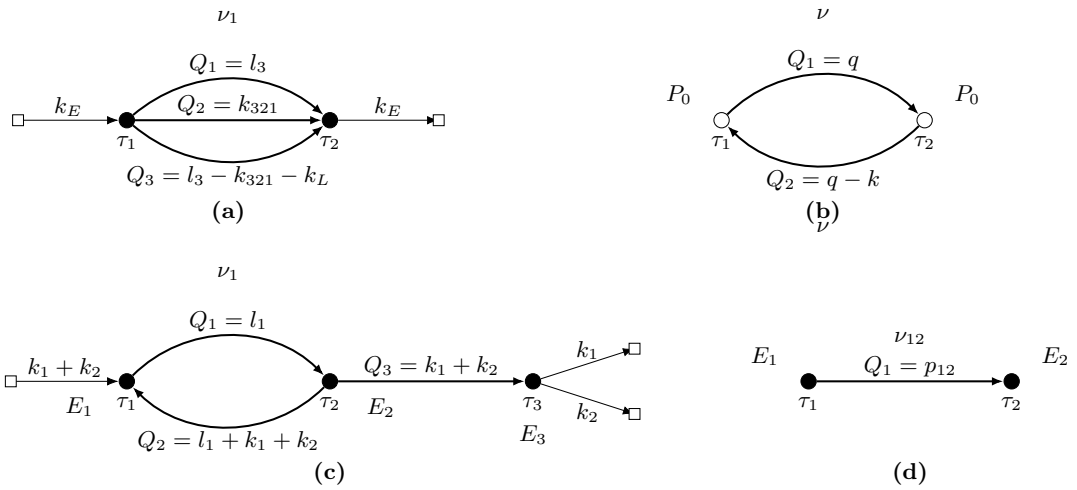


Figure 4. Example graphs. The fill colour of a vertex encodes its branch sign: black is + and white is −. (a) The two-loop sunrise of example 03: one massive full propagator of mass ν_1 and two massless full propagators between two + vertices; both vertices carry the same external-leg energy k_E. (b) The pure massive bubble of example 04: two massive full propagators of the same mass ν between two − vertices with vertex energy P_0; the loop external momentum is k. (c) The mixed bubble-plus-tree case of example 06: a massive full propagator of mass ν_1 and a massless full propagator form the loop between v_1 and v_2, and a massless bridge propagator connects v_2 and v_3; the external legs carry $k_1 + k_2$, k_1 and k_2. (d) The two-vertex tree of example 05: a massive propagator of mass ν_{12} whose momentum p_{12} is a no-loop magnitude.

receives two time generators and $L(L + K) = 6$ loop-momentum generators. Two ISPs are declared, $\mathrm{sp}[k_{321}, l_3]$ and $\mathrm{sp}[l_3 - k_{321} - k_L, l_3]$, each with range $\{0, 1\}$. The symmetry that swaps the two massless propagators exchanges them, and the package canonicalizes the two ISPs together with that swap. The example stops after the general stage. It generates the canonical seeds and the general parameter-derivative operators, and it neither samples the continuous labels nor runs a reduction, a differential equation or a scaling check. The seeds are checked by applying `ds` to the top-sector integral with respect to both variables.

Pure massive bubble (example 04). The bubble of figure 4b has two vertices on the − branch. Its two massive full propagators carry the same mass ν and the loop momenta q and $q-k$. The loop external momentum k gives the loop coordinate $\mathtt{ss11} = |k|$. Both vertex energies are set to the parameter P_0, which is the second derivative variable. This example runs the full closed loop: seeds, envelope sweep, linear system, two-stage Kira export, external reduction, import, differential equations and scaling check. The Kira workspace must lie outside the repository, and the package itself never starts Kira. The linear system is written at a fixed rational probe point for the dimension and the masses, while the differential equations keep `ss11` and P_0 symbolic. The reduction closes with no unreduced target and returns nineteen master integrals. The two 19×19 differential-equation matrices in $d\log$ form reproduce those of ref. [2]: at the recorded exact point each of the three variable conventions agrees entry by entry, 361/361. The scaling identities (3.2) hold symbolically

on the same basis, with 33/33 source-level identities and 19/19 active-basis checks.

Mixed bubble plus tree (example 06). The most demanding released case is figure 4c. Propagators 1 and 2 form the loop: propagator 1 is a massive full propagator of mass ν_1, and propagator 2 is a massless full propagator. Propagator 3 is a massless bridge between v_2 and v_3. It lies outside the loop, so its pack carries no denominator power. The external legs carry k_1+k_2 into v_1 and k_1, k_2 into v_3. The loop external momentum is k_1+k_2, which gives `ss11`. The independent no-loop magnitudes k_1, k_2 give `sE1`, `sE2`. Both cycle propagators are restricted to even parity, $b_{[e]}+n_{[e,1]}+n_{[e,2]} \equiv 0$ for $e = 1, 2$, while the bridge propagator is excluded from the parity constraints. The parity-restricted family has eighty-one master integrals, and the generated system contains 818,217 relations. The external Kira reduction takes about 160 s and closes with no unreduced target. The package then produces six 81×81 differential-equation matrices, one for each of the variables `ss11`, `sE1`, `sE2`, E_1, E_2, E_3, with no residual symbols. All 81/81 source-level Euler and IBP identities vanish exactly. The reduction-based scaling certificate is evaluated at the recorded exact point, because all six scaling variables are fixed in the formal plan. The example also exercises the parameter redefinition `DSRedefineParameters`, a bound-energy coordinate binding $E_0 = |k_1| + |k_2|$, and the over-complete and under-complete kinematic audits.

Two-vertex tree (example 05). A massive propagator of mass ν_{12} connects two + vertices, figure 4d. There is no loop momentum. The momentum p_{12} of this propagator is declared as an independent no-loop magnitude, so the package builds the coordinate `sE1` $= |p_{12}|$. The `timeOnly` family has the two sector keys `"1"` and `"0"`, for the full and the shrunk propagator. The iterative route and the naive IBP route give the same $d\log$ differential equations in E_1, E_2, `sE1`, so `deRoutesAgree` is `True`. The same example also declares a massless family with a single propagator. Its quotient states reduce to $n_{1,1} \to 0$, $n_{1,1} \to 1$ and $n_{1,2} \to 0$ over the sector keys `"0"` and `"1"`. Its formula route returns `PendingRederivation` with the reason `masslessQuotientFormulaNotCertified`, as required by the gate of section 2.6.

Mixed bubble workflow and bare Hankel system (examples 01, 02). Example 01 declares a bubble with one massive and one massless full propagator between two + vertices. It runs the unified public entry through the discrete templates, the continuous sweep and the linear system. It builds the pre-reduction plan in memory without exporting or running Kira. Example 02 supplies a bare Hankel function system (P, Q, T, W) instead of the `"h"` preset. The package compiles it through the same public workflow and generates the seeds of the corresponding family. Together the two examples cover the initialization, the envelope sweep and the function-system interface of sections 3 and 2.5.

5 Summary and outlook

`dSIBP` turns the canonical IBP structure of de Sitter time integrals into a practical MATHEMATICA tool. From a graph declaration it produces sealed canonical seeds, backend-neutral

linear systems, Kira and Rational-Tracer export manifests with artifact identities, differential equations and scaling checks. It accepts arbitrary topologies that mix massive and massless propagators of all four Schwinger–Keldysh types. The parity-split structure, the massive equation-of-motion reduction and the massless quotient are applied by construction, so the user never handles raw label spaces. Ongoing work concerns the certification of the tree-level formulas for families that contain massless full propagators, which are currently gated behind `PendingRederivation`. Further directions are a deeper integration with hypergeometric solution methods [20] and the connection to numerical evaluation pipelines such as MadStree [21] and to high-precision series-transport solvers [17].

Acknowledgements

We thank the developers of Kira [12, 13], Rational Tracer [14] and MadStree [21] for making their tools available.

A Public functions

Function	Description
`DSInit`	initialize a case; structural and kinematic audits; returns the context.
`DSKinematics`	two-stage kinematic proposal and validation.
`DSParameterNotation`	inspect the current parameter notation.
`DSRedefineParameters`	replace the dynamical rules; re-audits and rehashes.
`DSSeeds`	canonical seed templates of a family.
`DSAllSeeds`	flattened seed template list.
`DSSeedGroups`, `DSSeedGroupMetadata`	grouping of the templates by sector, class and generator.
`DSMetaSeedRange`	per-group shift metadata for envelope planning.
`DSGenerateIBP`	sweep continuous indices; sealed IBP relations.
`DSLinear`	backend-neutral linear system (`linearData`).
`DSReorderIntegrals`	re-index the master-integral ordering.
`DSKiraPlan`	in-memory pre-reduction or formal reduction plan.
`DSKiraExport`	write Kira input directories; manifest with SHA-256 identities.
`DSKiraImport`	read back external reduction results with identity checks.
`DSUserMI`	attach a user master-integral basis with rank checks.
`DSDE`	differential equations of the reduced basis.
`DSScaleCheck`	Euler scaling identities of the family.
`DSTreeSeeds`	`timeOnly` seed templates.
`repIterative`	tree route via the general derived formulas.
`DSTreeNaiveIBP`, `DSTreeNaiveDE`	tree route via raw sampled IBP.
`DSTreeDLogDE`	tree $d\log$ differential equations.
`dtau`, `dqq`, `dqk`, `ds`	atomic generators and the full variable derivative.
`rep2innerform`, `rep2outform`	notation rewriters between user and internal forms.
`rep2Integrand`	expand J into the explicit integrand.
`symmetry`	apply user and gated self-loop symmetry rules.

References

[1] X. Chen, Y. Wang and Z.-Z. Xianyu, *Schwinger-Keldysh Diagrammatics for Primordial Perturbations*, *JCAP* **12** (2017) 006 [1703.10166].

[2] J. Chen, B. Feng, Z. Qin and Y.-X. Tao, *Loop integrals in de Sitter spacetime: The parity-split IBP system and* d log*-form differential equations*, 2604.14549.

[3] K.G. Chetyrkin and F.V. Tkachov, *Integration by parts: The algorithm to calculate β-functions in 4 loops*, *Nucl. Phys. B* **192** (1981) 159.

[4] S. Laporta, *High-precision calculation of multiloop Feynman integrals by difference equations*, *Int. J. Mod. Phys. A* **15** (2000) 5087 [hep-ph/0102033].

[5] A.V. Kotikov, *Differential equations method: New technique for massive Feynman diagrams calculation*, *Phys. Lett. B* **254** (1991) 158.

[6] A.V. Kotikov, *Differential equation method: The Calculation of N point Feynman diagrams*, *Phys. Lett. B* **267** (1991) 123.

[7] Z. Bern, L.J. Dixon and D.A. Kosower, *Dimensionally regulated pentagon integrals*, *Nucl. Phys. B* **412** (1994) 751 [hep-ph/9306240].

[8] T. Gehrmann and E. Remiddi, *Differential equations for two-loop four-point functions*, *Nucl. Phys. B* **580** (2000) 485 [hep-ph/9912329].

[9] J.M. Henn, *Multiloop integrals in dimensional regularization made simple*, *Phys. Rev. Lett.* **110** (2013) 251601 [1304.1806].

[10] A.V. Smirnov and F.S. Chukharev, *FIRE6: Feynman Integral REduction with modular arithmetic*, *Comput. Phys. Commun.* **247** (2020) 106877 [1901.07808].

[11] A.V. Smirnov and M. Zeng, *FIRE 7: Automatic Reduction with Modular Approach*, *Comput. Phys. Commun.* **325** (2026) 110200 [2510.07150].

[12] P. Maierhöfer, J. Usovitsch and P. Uwer, *Kira—A Feynman integral reduction program*, *Comput. Phys. Commun.* **230** (2018) 99 [1705.05610].

[13] J. Klappert, F. Lange, P. Maierhöfer and J. Usovitsch, *Integral reduction with Kira 2.0 and finite field methods*, *Comput. Phys. Commun.* **266** (2021) 108024 [2008.06494].

[14] V. Magerya, *Rational Tracer: a Tool for Faster Rational Function Reconstruction*, 2211.03572.

[15] M. Hidding, *DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions*, *Comput. Phys. Commun.* **269** (2021) 108125 [2006.05510].

[16] X. Liu, Y.-Q. Ma and C.-Y. Wang, *A Systematic and Efficient Method to Compute Multi-loop Master Integrals*, *Phys. Lett. B* **779** (2018) 353 [1711.09572].

[17] X. Liu and Y.-Q. Ma, *AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow*, *Comput. Phys. Commun.* **283** (2023) 108565 [2201.11669].

[18] Z.-F. Liu and Y.-Q. Ma, *Determining Feynman Integrals with Only Input from Linear Algebra*, *Phys. Rev. Lett.* **129** (2022) 222001 [2201.11637].

[19] J. Chen and B. Feng, *Towards systematic evaluation of de Sitter correlators via Generalized Integration-By-Parts relations*, *JHEP* **06** (2024) 199 [2401.00129].

[20] J. Chen, B. Feng and Y.-X. Tao, *Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations*, *JHEP* **03** (2025) 075 [2411.03088].

[21] "MadStree: a package for quickly evaluating arbitrary tree-level cosmological correlator." in preparation, 2026.

[22] D. Baumann, H. Goodhew and H. Lee, *Kinematic flow for cosmological loop integrands*, *JHEP* **07** (2025) 131 [2410.17994].

[23] B. Fan and Z.-Z. Xianyu, *Cosmological amplitudes in power-law FRW universe*, *JHEP* **12** (2024) 042 [2403.07050].

[24] Y. Hang and C. Shen, *A note on kinematic flow and differential equations for two-site one-loop graph in FRW spacetime*, *JHEP* **09** (2025) 209 [2410.17192].